Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #1| Lenguajes de programación 3

Ingeniería en Desarrollo de Software



TUTOR: Miguel Angel Rodríguez Vega

ALUMNO: Edgar Enrique Cuamea Ochoa

FECHA: 21 de julio del 2025

Contenido

Introducción. 3

Descripcion 4

Justificación. 5

Desarrollo. 6

Conclusión. 23

Referencias 24

Introducción.

Realizaremos una base de datos para el desarrollo de la actividad, en este caso realizaremos un ABC, Alta, Baja y Consulta de empleados para una empresa mexicana, la base de datos contiene diferentes datos de los empleados así como diferentes empleados donde diferenciamos a los directivos de los que no lo son además de indicar el centro donde trabajan a través de una tabla donde se pueda consultar la información de los empleados en la base de datos, este contendrá numero de centro, nombre del centro y ciudad donde está ubicada, en este sistema podemos dar de alta a diferentes empleados ingresando los datos que se solicitan además de poder realizar consultas de los diferentes empleados, de esta forma podremos revisar la información de cada empleado, además debemos poder dar de baja a algún empleado en el sistema por lo que es necesario poder conectar el programa que realizaremos en visual studio a una base de datos por medio de SQL.

Descripcion.

Crearemos nuestra base de datos utilizando SQL management studio por lo que es necesario tener los diagramas de entidad relación, así como el diagrama lógico-relacional para tener claro la estructura de la base de datos que crearemos en nuestro sistema local. Además de como definir las llaves primarias y foráneas, además de definir las clases así como las relaciones de los diferentes datos que ingresaremos como las llaves primarias y llaves foráneas además de tener diferentes datos como nombres, números de empleado, dirección, etc., teniendo en cuenta los requisitos que veremos en el desarrollo de la actividad, de esta forma podemos crear nuestra base de datos funcional para después vincularla con el programa que realizaremos en visual studio utilizando el lenguaje de C#, de esta manera realizaremos un programa donde se puedan dar de alta a empleados, realizar bajas de empleados y consultas al establecer una conexión con nuestra base de datos.

Justificación.

Realizaremos la base de datos en SQL management studio, este programa nos permite realizar bases de datos en nuestra computadora de manera local, en actividades anteriores trabajamos utilizando este programa, además de poder realizar nuestra base de datos podremos vincularla a visual studio, este en un programa que nos brinda herramientas para la programación como distintos lenguajes de programación, funciones dentro de este, compiladores así como poder realizar el vinculo con la base de datos, de esta forma podemos crear aplicaciones de escritorio utilizando diferentes lenguajes de programación, en este cao usaremos C# para crear una aplicación de escritorio con un entorno grafico donde podamos vincularla a la base de datos creada para así poder realizar consultas, alta y bajas de distintos empleados en la base de datos, de esta forma tendremos mucha libertad al momento e crear aplicaciones, ya sean de escritorio Windows, MacOS, XBOX, Android, Linux, etc por lo que el lenguaje de C# ofrece distintas plataformas de desarrollo.

Desarrollo.

Realizaremos nuestra base de datos en SQL management studio por lo que realizamos un análisis de la siguiente información

Número de empleado	Autoincrementable, autogenerado, numérico ID
Nombre	Capturable, alfanumérico
Apellido Paterno	Capturable, alfanumérico
Apellido Materno	Capturable, alfanumérico
Fecha de Nacimiento	Capturable, tipo *date* (fecha)
RFC	
Centro de trabajo	Capturable, alfanumérico
Puesto	Llave foránea de la tabla Catálogo de Puestos
Descripción del puesto	Desplegar según el puesto elegido en la llave foránea
Directivo	Bandera que indica 1) para Directivo y 0) para empleado normal

● Existe un tipo de empleado denominado Directivo, el cual presenta, además de las cualidades anteriores, algunas atribuibles solo para este tipo de empleado. Las características específicas del Directivo son: o Centro supervisado (numérico, capturable) o Prestación de combustible (bandera que indica si el directivo recibe apoyo de combustible)

Número de Centro	**Nombre de Centro**	**Ciudad**
000201	Tiendas Ángel Flores Ropa	Culiacán
000202	Tiendas Ángel Flores Muebles	Culiacán
000203	Tiendas Ángel Flores Cajas	Culiacán
049001	La Primavera Ropa	Culiacán
049002	La Primavera Muebles	Culiacán
049003	La Primavera Cajas	Culiacán

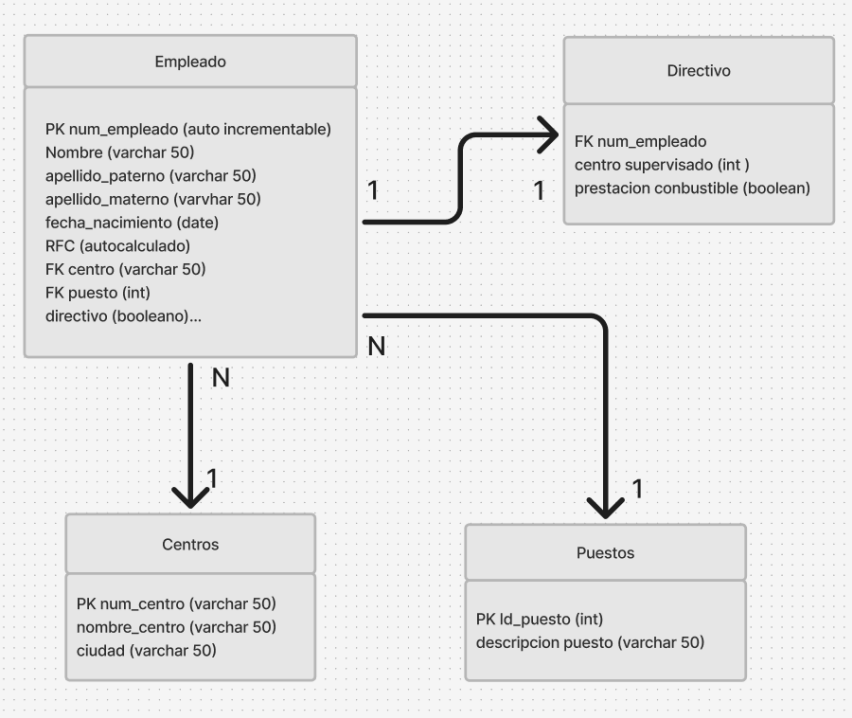
Para la opción de **altas**, permitir al usuario la captura de todos los campos *capturables*; mientras que los campos calculados, como el RFC o descripciones de catálogos, deben desplegarse una vez capturados los datos para que el cálculo sea posible.

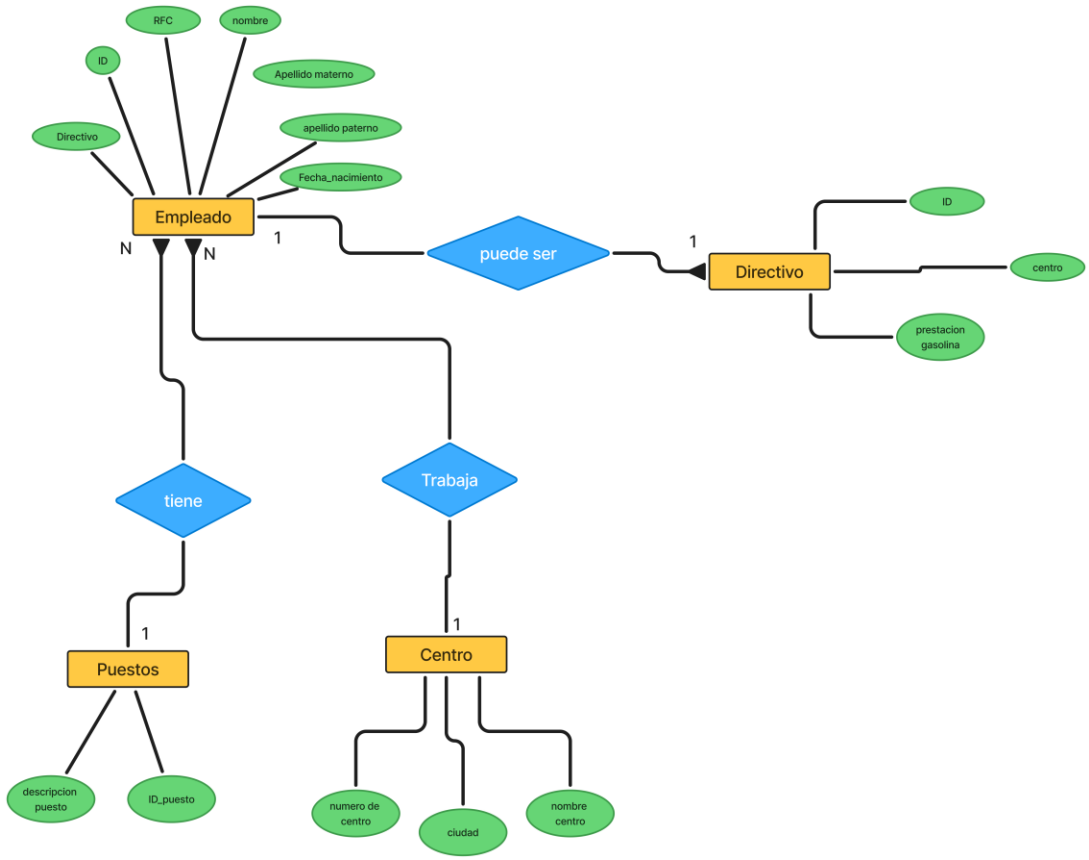
Para el módulo de **bajas**, capturar el número de empleado, y una vez desplegados los datos, se debe permitir la opción para eliminarlos.

Para el módulo de **cambios o actualización**, capturarse el número de empleado. Una vez desplegados los datos, debe permitir cambios en los datos editables. Después de realizar las modificaciones, dar la opción de guardar estos cambios.
Por su parte, la opción de reporte de empleados debe respetar el siguiente *layout*:

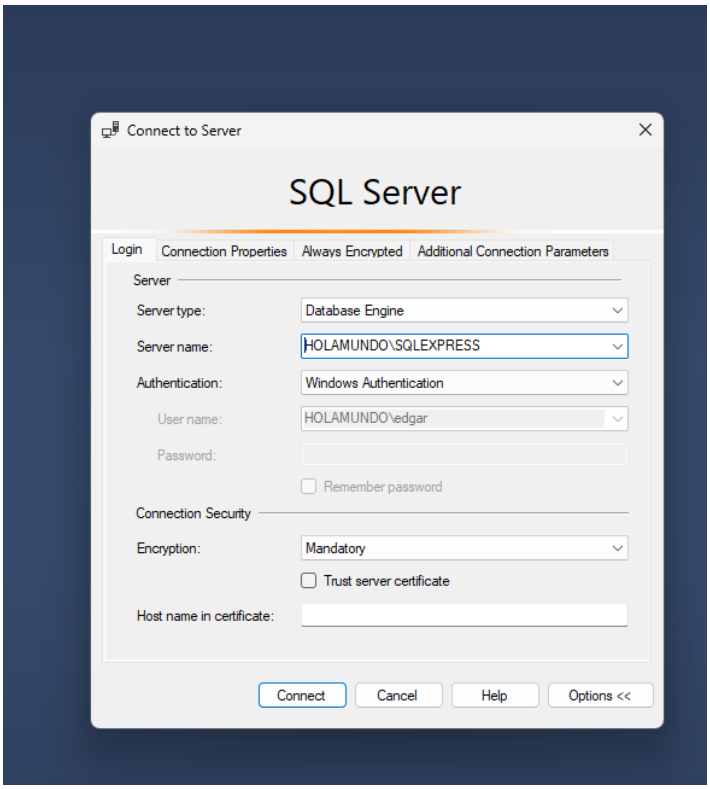
Numero de Empleado	Nombre + Apellido Paterno + Apellido Materno	Fecha de Nacimiento	RFC	Nombre de Centro	Descripción del Puesto	¿Es Directivo?
1	Jesús Vega Castro	26/03/1988	VECJ880326XXX	Tiendas Ángel Flores Ropa	Vendedor	No
2	José López Perez	01/01/1980	LOPJ800101XXX	La Primavera Cajas	Gerente	Si
3	.....	.....	.....	.....	.....	.....

Una vez que realizamos el análisis de esta información, realizaremos nuestros diagramas de entidad relación, así como el diagrama lógico relacional por lo que utilizaremos el software de figma para la realización de los diagramas:

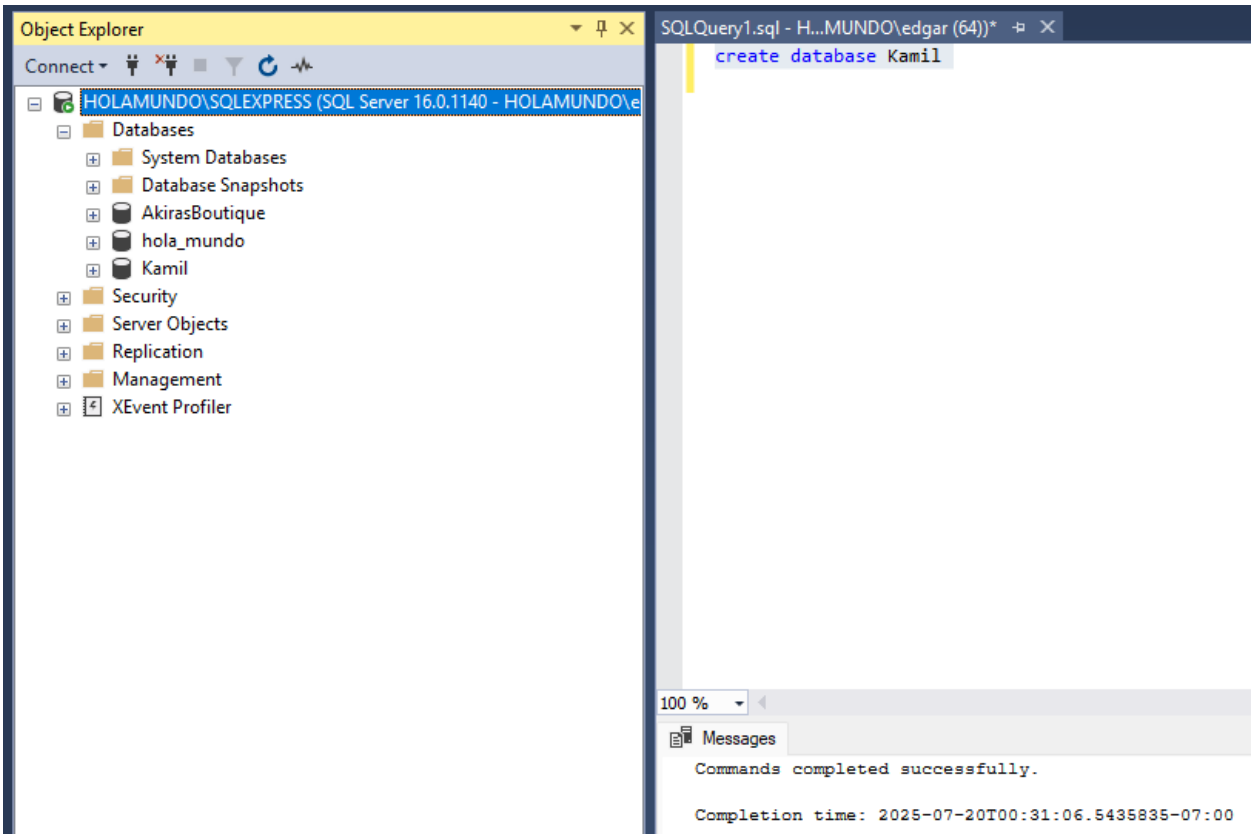

Empleado
PK num_empleado (auto incrementable)
Nombre (varchar 50)
apellido_paterno (varchar 50)
apellido_materno (varvhar 50)
fecha_nacimiento (date)
RFC (autocalculado)
FK centro (varchar 50)
FK puesto (int)
directivo (booleano)...
Directivo
FK num_empleado
centro supervisado (int)
prestacion conbustible (boolean)
1
1
N
N
1
1
Centros
PK num_centro (varchar 50)
nombre_centro (varchar 50)
ciudad (varchar 50)
Puestos
PK Id_puesto (int)
descripcion puesto (varchar 50)


RFC
nombre
ID
Apellido materno
Directivo
apellido paterno
Fecha_nacimiento
Empleado
N
N
1
puede ser
1
Directivo
ID
centro
prestacion gasolina
tiene
Trabaja
1
1
Puestos
Centro
descripcion puesto
ID_puesto
numero de centro
ciudad
nombre centro

Una ves realizadas los diagramas entidad relación, así como el diagrama lógico relacional, identificamos las llaves primarias que utilizaremos para las distintas clases, estos contaran con diferentes atributos para cada uno, así como las llaves foráneas que estableceremos en la base de dato por lo que, una vez identificados las clases y atributos y relaciones, abriremos SQL management studio para crear la nueva base de datos



Nos conectaremos a nuestra base de datos local a través de la autenticación de Windows y veremos nuestras bases de datos que tenemos utilizando este software por lo que creamos nuestra base de datos nueva



Creamos una nueva base de datos llamada Kamil, en esta nueva base de datos crearemos las tablas juntos con sus atributos, así como las relaciones con las demás tablas

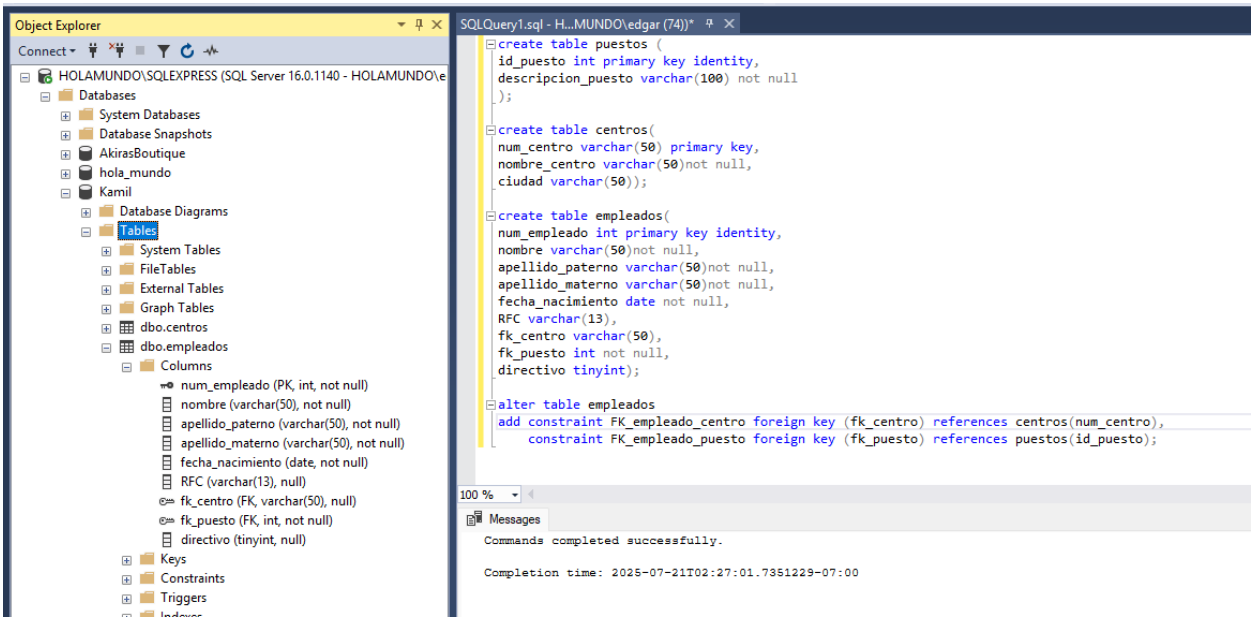
```
create table puestos (
id_puesto int primary key identity,
descripcion_puesto varchar(100) not null
);

create table centros(
num_centro varchar(50) primary key,
nombre_centro varchar(50)not null,
ciudad varchar(50));

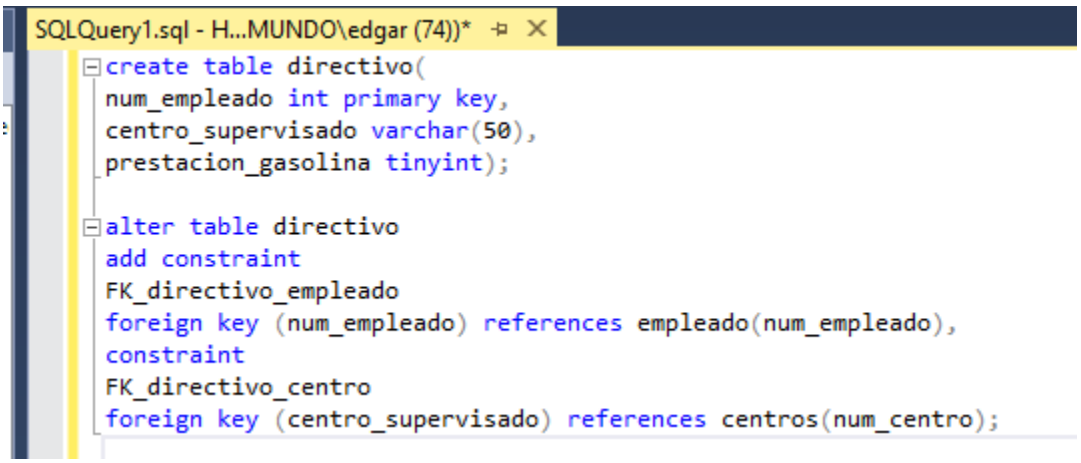
create table empleados(
num_empleado int primary key identity,
nombre varchar(50)not null,
apellido_paterno varchar(50)not null,
apellido_materno varchar(50)not null,
fecha_nacimiento date not null,
RFC varchar(13),
fk_centro varchar(50),
fk_puesto int not null,
directivo tinyint);
```

En este caso creamos las tablas con sus respectivas llaves primarias, tenemos la tabla puestos,

centros y empleados, cada una con sus atributos así como el tipo de dato que tiene cada atributo, en este caso utilizamos el tipo de dato tinyint en el atributo de directivo ya que este no admite el tipo booleano de manera nativa por lo que utilizamos este dato para almacenar un cierto o falso para determinar si el empleado es directivo, establecemos las llaves primarias y foráneas para la tabla de empleados con las llaves primarias de centro y puesto antes de agregar la tabla de directivo



Creamos nuestra tabla directiva junto con la llave primaria y foránea que verificara que el empleado sea directivo o no



Tenemos nuestra llave primaria que es el numero de empleado, seleccionamos las llaves foráneas

desde la tabla centro que será el centro supervisado, así como el numero de empleado de la tabla empleados en caso de ser directivo.

Una vez creada la base de datos, agregaremos los empleados, diferentes puestos y centros en la base de datos utilizando la información de las tablas presentadas al inicio del desarrollo por lo que realizamos lo siguiente

```
insert into centros (num_centro, nombre_centro, ciudad)
VALUES
    ('000201', 'Tiendas Ángel Flores Ropa', 'Culiacán'),
    ('000202', 'Tiendas Ángel Flores Muebles', 'Culiacán'),
    ('000203', 'Tiendas Ángel Flores Cajas', 'Culiacán'),
    ('049001', 'La Primavera Ropa', 'Culiacán'),
    ('049002', 'La Primavera Muebles', 'Culiacán'),
    ('049003', 'La Primavera Cajas', 'Culiacán');

    set identity_insert puestos on;
insert into puestos (id_puesto, descripcion_puesto)
values
    (1, 'Cajero'),
    (2, 'Vendedor'),
    (3, 'Directivo');
    set identity_insert puestos off;
```

```
%
Messages
Msg 544, Level 16, State 1, Line 24
Cannot insert explicit value for identity column in table 'puestos' when IDENTITY_INSERT is set to OFF.

Completion time: 2025-07-21T15:05:02.9580506-07:00
```

Vemos que aparece un error al momento de querer agregar los puestos por lo que al agregar la tabla de puestos, el id del puesto lo definimos como auto incrementable con el parámetro identity por lo que es necesario activar el identity_insert para que nos permita ingresar los id de manera manual por lo que lo activamos, agregamos los puestos y lo desactivamos , en este caso al agregar un nuevo puesto, la base de datos lo agregara con el id de 4 y autoincrementándose por cada puesto nuevo que agreguemos

```
insert into Empleados (nombre,apellido_paterno,apellido_materno,fecha_nacimiento,RFC,fk_centro,fk_puesto,directivo)
values (
    'Jesus','Vega','Castro','1968-03-26','VECJ880326XXX','000201',2,1),(
    'Jose','López','Perez','1980-01-01','LOPI800101XXX','049003',3,0);
```

```
Messages

(2 rows affected)
```

Agregamos a los dos empelados de la tabla, uno como directivo y otro como vendedor

```
select * from empleados
select * from directivo
```

121 %

Results | Messages

	num_empleado	nombre	apellido_paterno	apellido_materno	fecha_nacimiento	RFC	fk_centro	fk_puesto	directivo
1	1	Jesus	Vega	Castro	1968-03-26	VECJ880326XXX	000201	2	1
2	2	Jose	López	Perez	1980-01-01	LOPI800101XXX	049003	3	0

	num_empleado	centro_supervisado	prestacion_gasolina

En este caso solo actualizamos una sola tabla por lo que es necesario crear un disparador donde si agregamos a un empleado y este esté marcado como directivo, lo debe de agregar a la tabla de directivo por lo que realizamos el siguiente disparador

```
create trigger
tgr_directivo
on empleados
after insert
as
begin
insert into directivo (num_empleado, centro_supervisado, prestacion_gasolina)
select I.num_empleado, I.fk_centro, 1
    from inserted as I
    where I.directivo = 1;
    end;

insert into Empleados (nombre,apellido_paterno,apellido_materno,fecha_nacimiento,RFC,fk_centro,fk_puesto,directivo)
values ('Jesus','Vega','Castro','1968-03-26','VECJ880326XXX','000201',2,1)


select * from empleados
select * from directivo
```

100 %

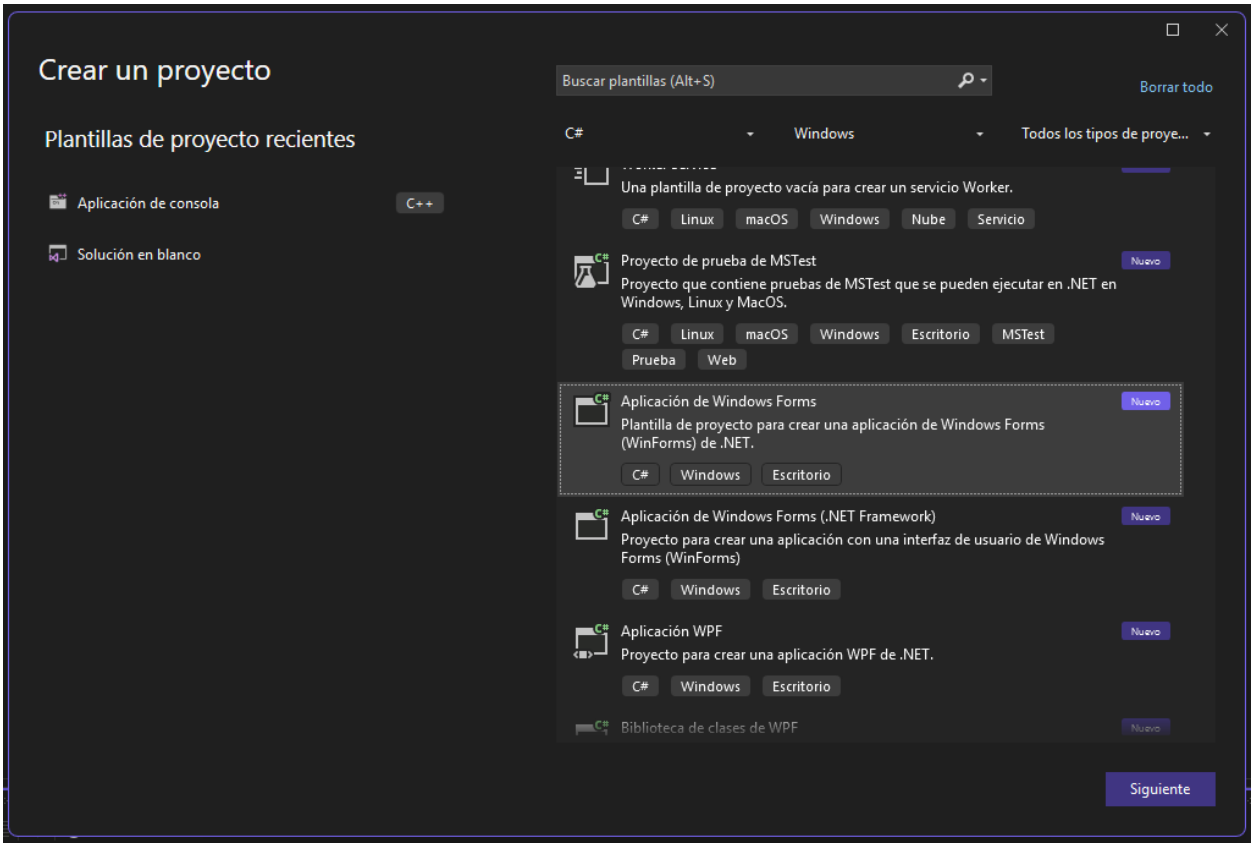
Results | Messages

	num_empleado	nombre	apellido_paterno	apellido_materno	fecha_nacimiento	RFC	fk_centro	fk_puesto	directivo
1	1	Jesus	Vega	Castro	1968-03-26	VECJ880326XXX	000201	2	1
2	2	Jose	López	Perez	1980-01-01	LOPI800101XXX	049003	3	0
3	3	Jesus	Vega	Castro	1968-03-26	VECJ880326XXX	000201	2	1

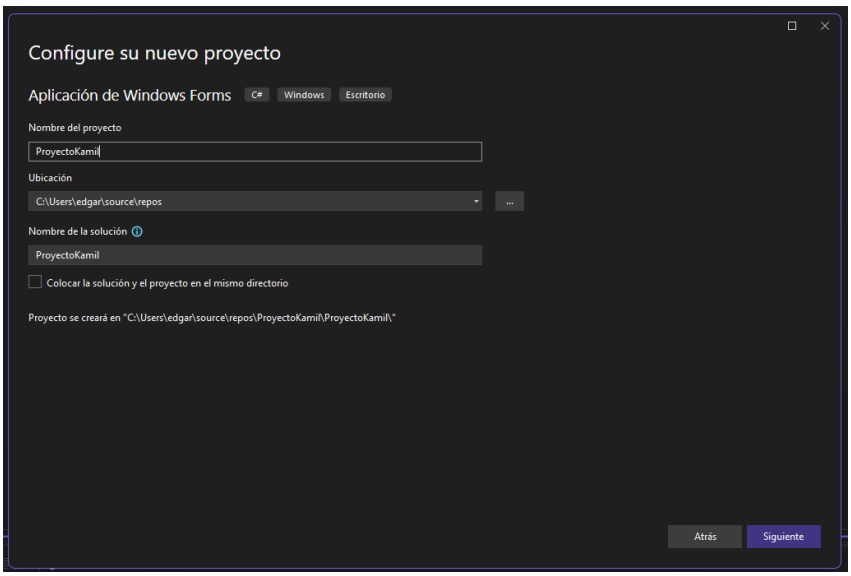
	num_empleado	centro_supervisado	prestacion_gasolina
1	3	000201	1

El disparador funciona de la siguiente manera, este código crea un disparador llamado tgr_directivo en la tabla empleados, este se activara después de insertar algún dato en empleados, una vez que se inserte un dato empleado insertara los datos de la tabla directivo y seleccionara el numero de empleado de la llave primaria de empleados así como el numero de centro de la llave foránea de centro insertándolas como una variable I, donde si el dato ingresado en la tabla directivo es si o 1, entonces ingresara los datos en la tabla directivo, si el dato es falso o 0 entonces no lo ingresara.

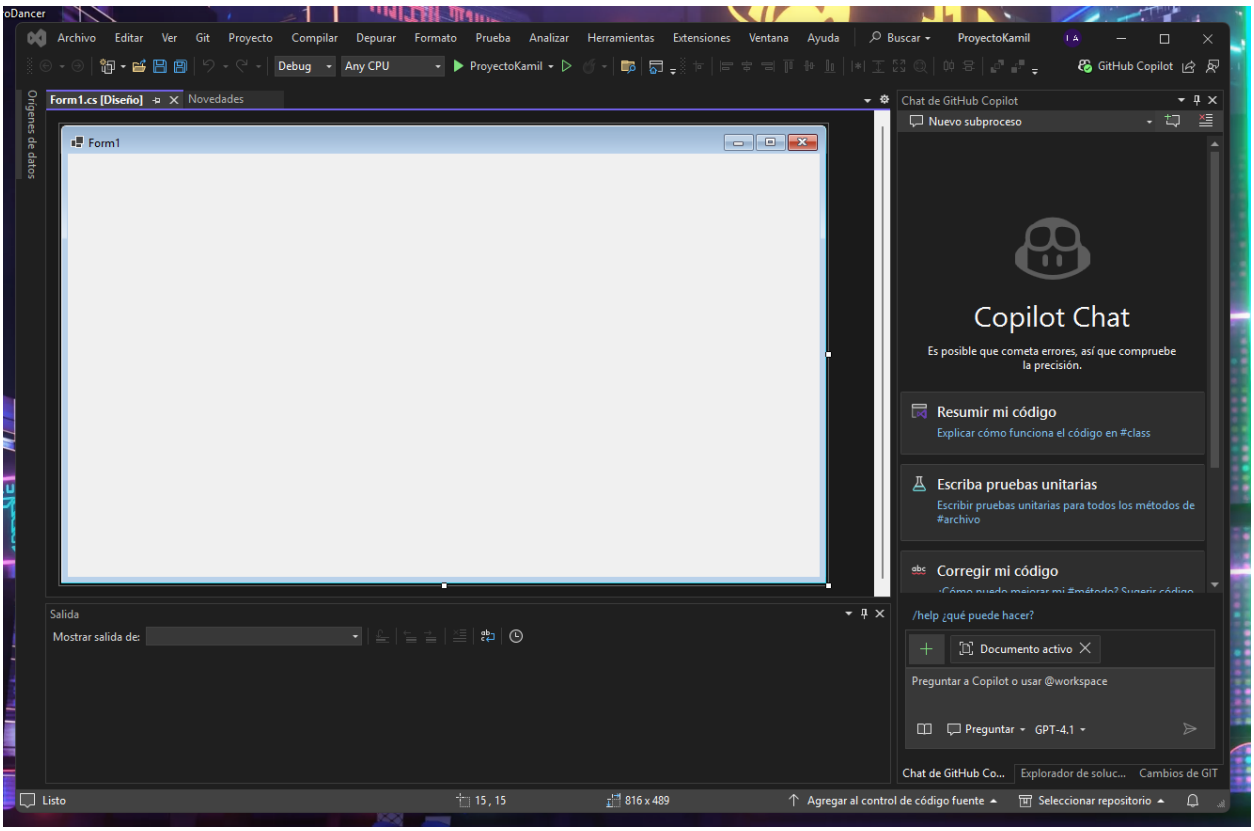
Una vez creada la base de datos en SQL y verificar que los centros, así como puestos estén ingresados, abriremos nuestro proyecto en visual studio para conectarnos a la base de datos utilizando el lenguaje c# en Windows forms



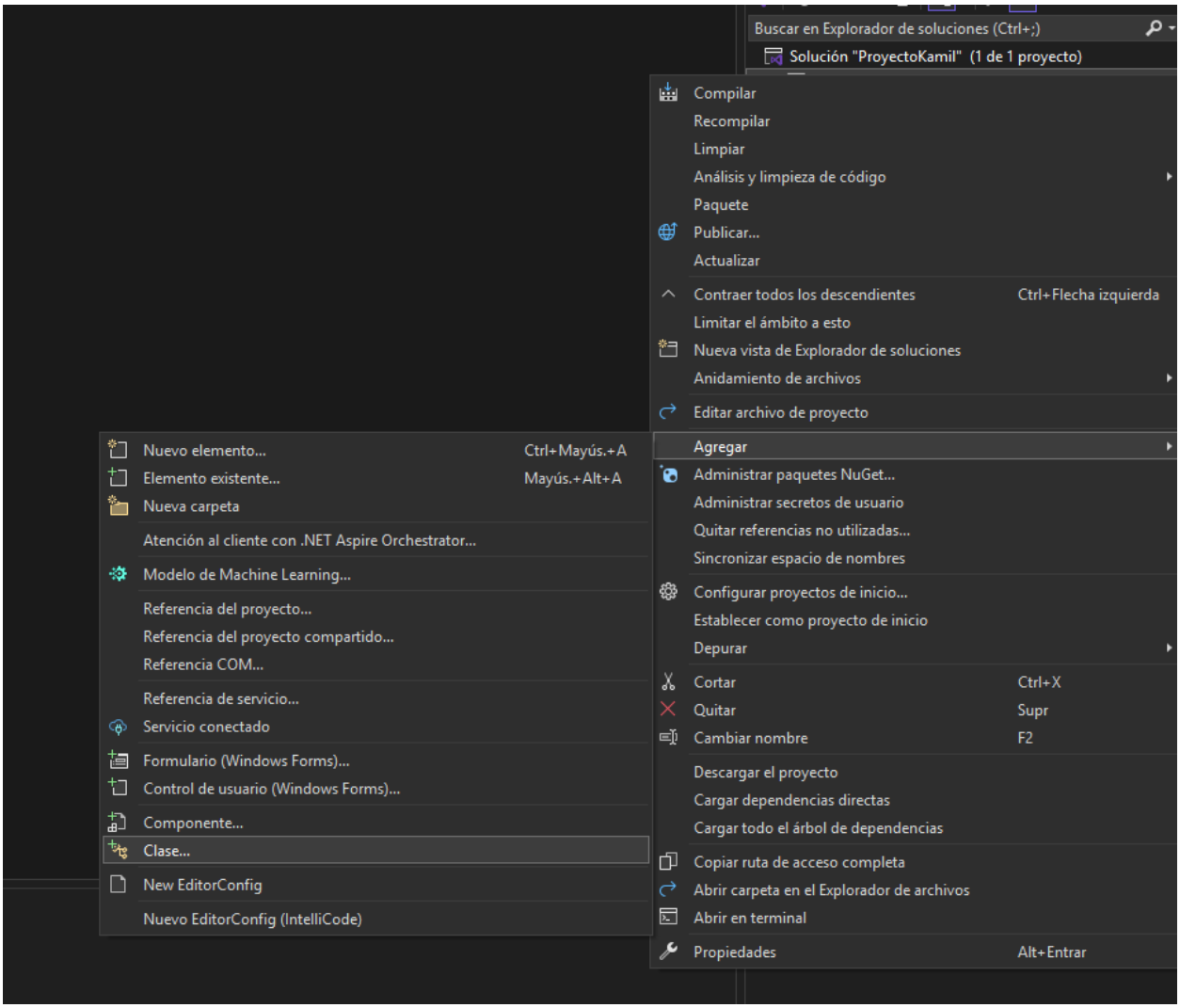
Creamos un proyecto nuevo donde realizaremos la conexión con la base de datos, en este caso crearemos una aplicación de Windows forms y utilizaremos el lenguaje de c#

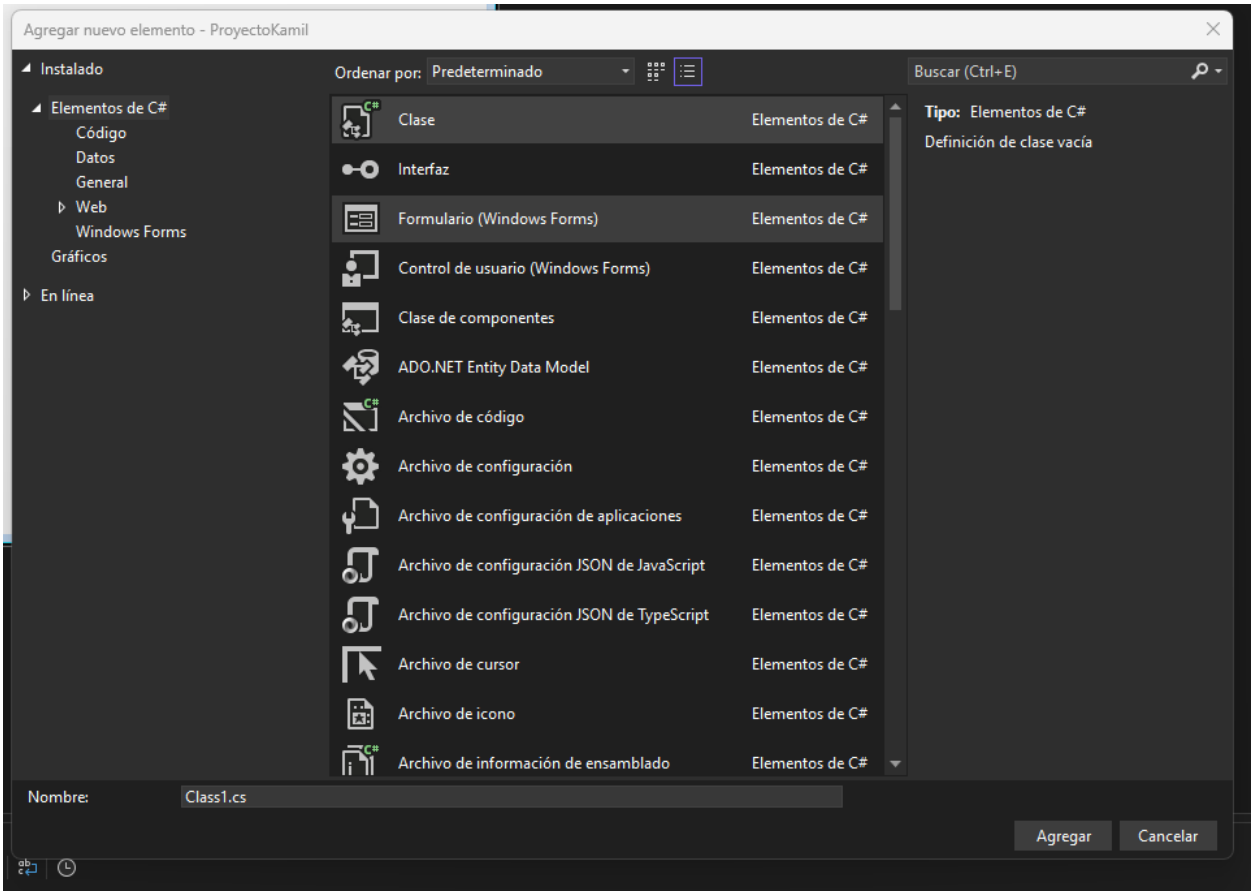


Creamos nuestro proyecto nombrado Proyecto Kamil

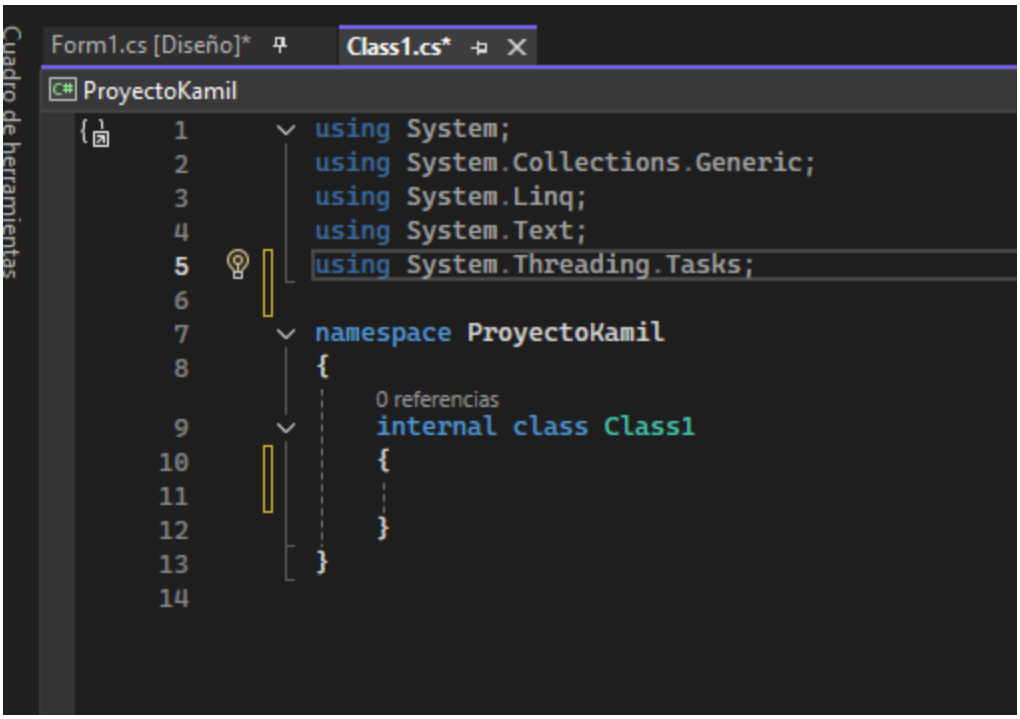


Y se creara la ventana de Windows forms, agregamos una clase para poder realizare la conexión con la base de datos

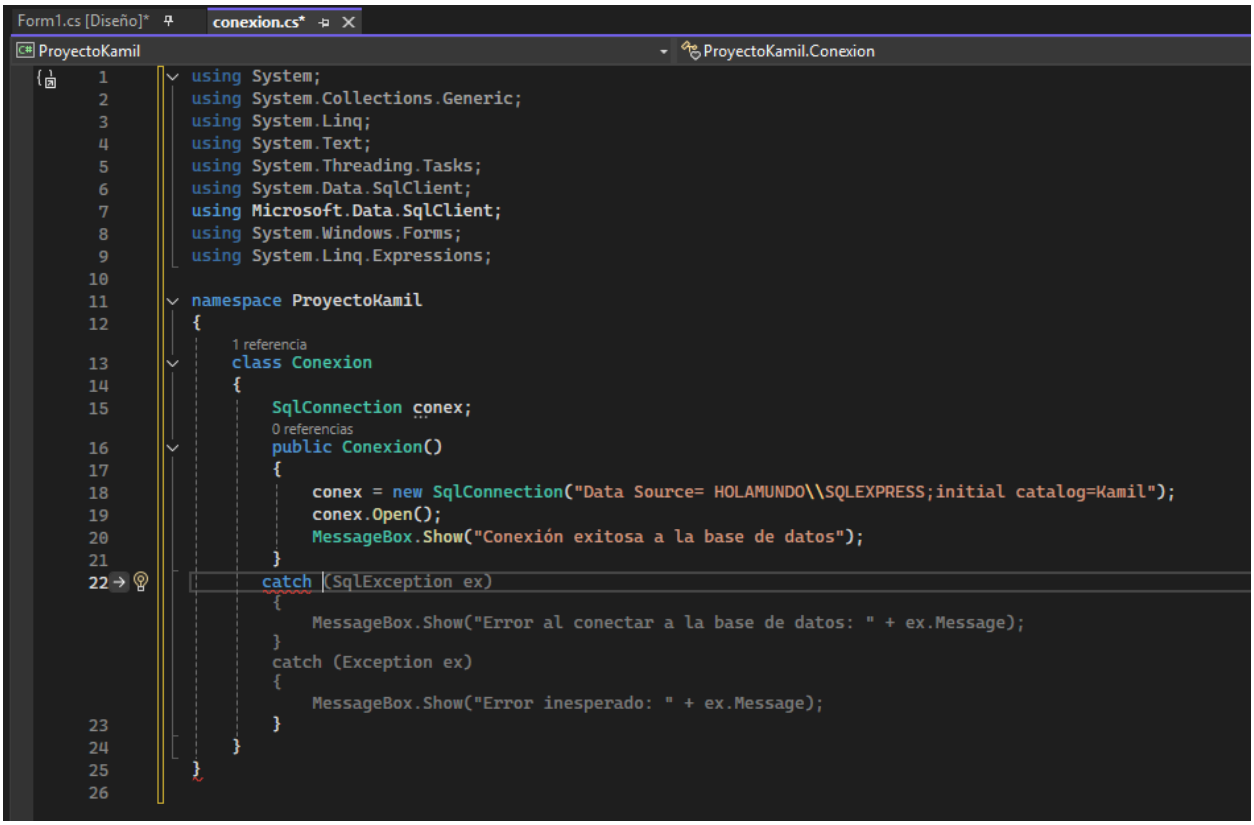

Buscar en Explorador de soluciones (Ctrl+;)
Solución "ProyectoKamil" (1 de 1 proyecto)
Compilar
Recompilar
Limpiar
Análisis y limpieza de código
Paquete
Publicar...
Actualizar
Contraer todos los descendientes Ctrl+Flecha izquierda
Limitar el ámbito a esto
Nueva vista de Explorador de soluciones
Anidamiento de archivos
Editar archivo de proyecto
Agregar
Administrar paquetes NuGet...
Administrar secretos de usuario
Quitar referencias no utilizadas...
Sincronizar espacio de nombres
Configurar proyectos de inicio...
Establecer como proyecto de inicio
Depurar
Cortar Ctrl+X
Quitar Supr
Cambiar nombre F2
Descargar el proyecto
Cargar dependencias directas
Cargar todo el árbol de dependencias
Copiar ruta de acceso completa
Abrir carpeta en el Explorador de archivos
Abrir en terminal
Propiedades Alt+Entrar
Nuevo elemento... Ctrl+Mayús.+A
Elemento existente... Mayús.+Alt+A
Nueva carpeta
Atención al cliente con .NET Aspire Orchestrator...
Modelo de Machine Learning...
Referencia del proyecto...
Referencia del proyecto compartido...
Referencia COM...
Referencia de servicio...
Servicio conectado
Formulario (Windows Forms)...
Control de usuario (Windows Forms)...
Componente...
Clase...
New EditorConfig
Nuevo EditorConfig (IntelliCode)



En este caso seleccionamos una clase vacía y le asignamos un nombre por lo que agregamos la clase

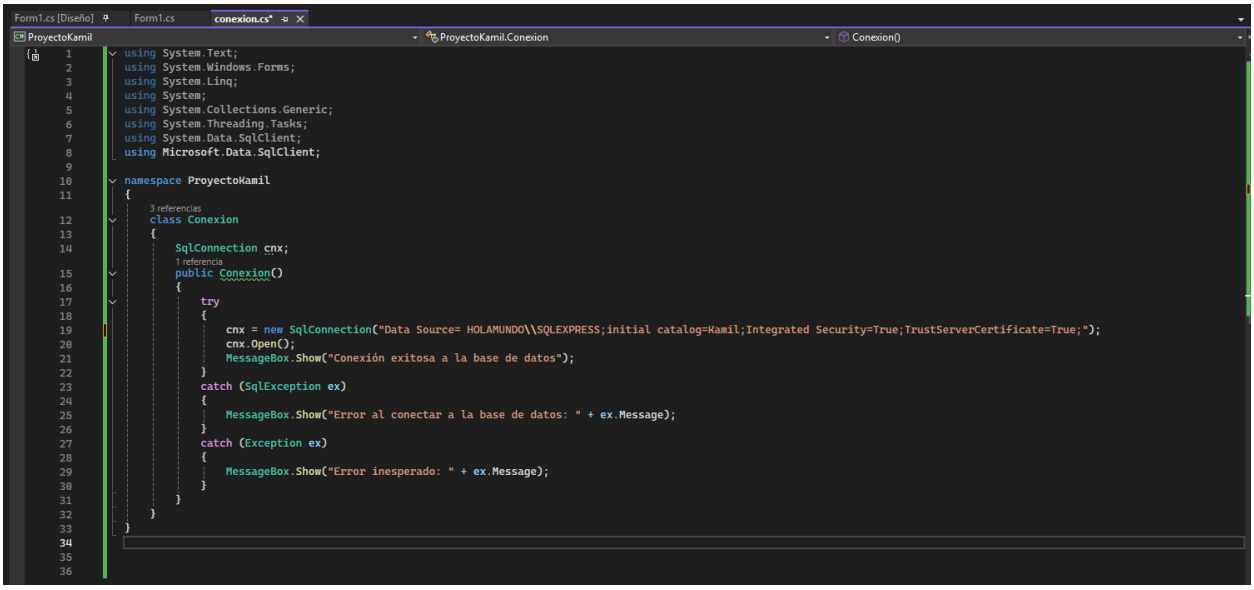


En este caso agregaremos el siguiente código para utilizar las librerías de SQL

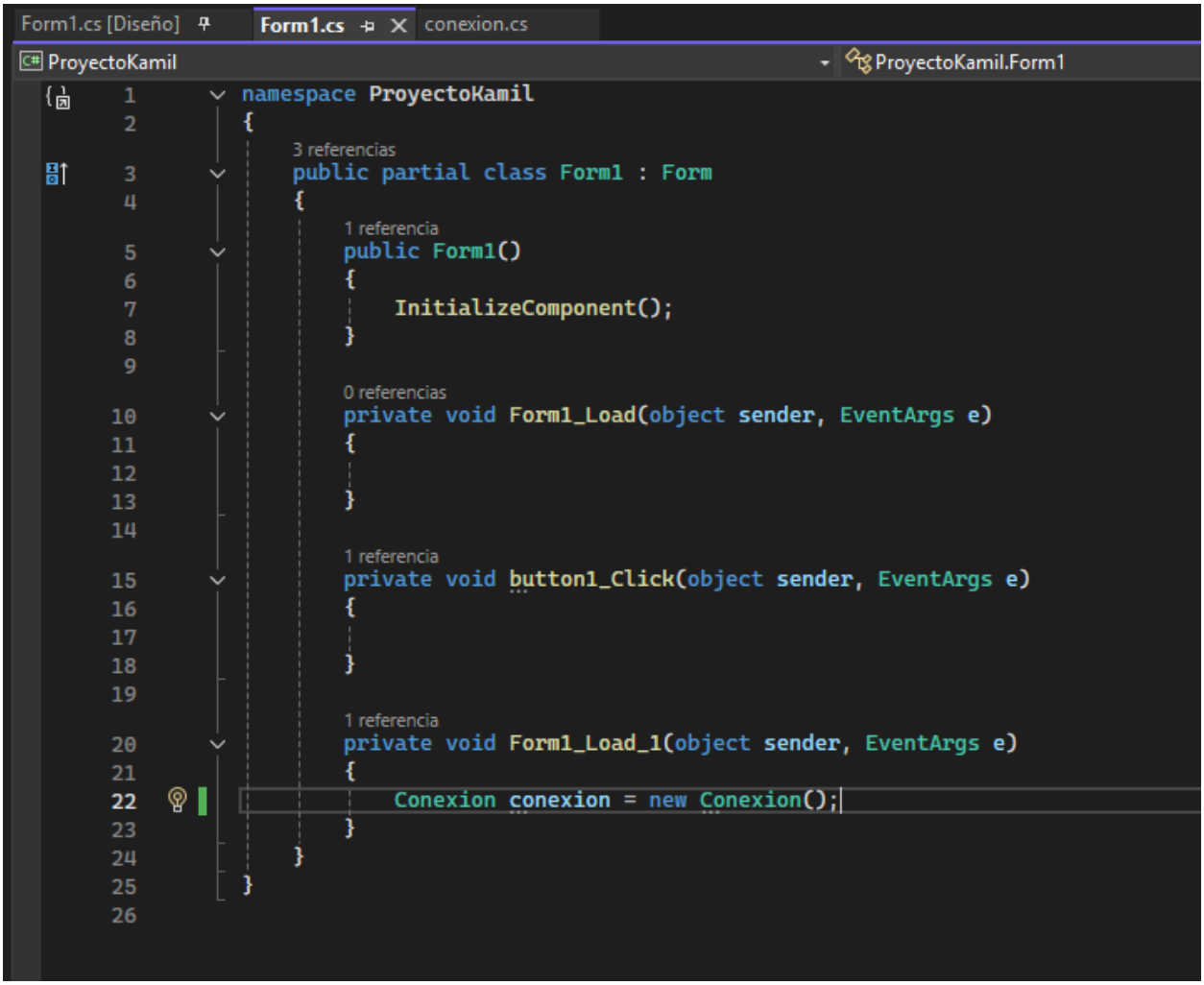


En este caso visual studio nos permite agregar varias líneas de código con un botón analizando

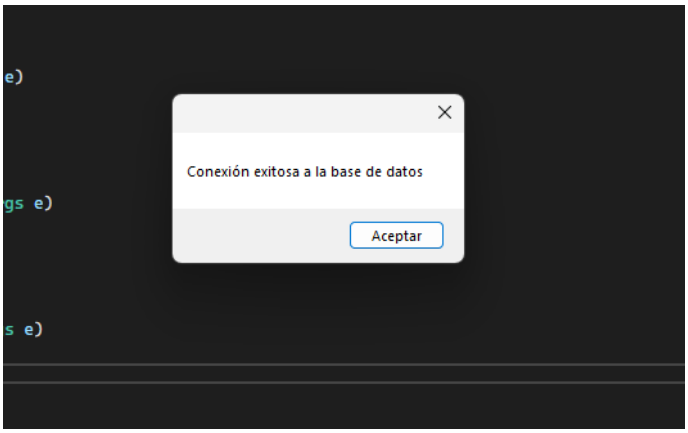
nuestro código y sugiriendo código para utilizar, en este caso el código sugerido sirve para mostrar en la pantalla que hay errores de conexión en caso de no conectarse a SQL



Agregamos un comando extra a la línea 19 donde establecemos que confiamos en el certificado de veracidad del servidor sql en visual studio ya que, sin este, si llega establecer conexión con la base de datos, pero da error al verificar las credenciales por lo que una vez realizado lo siguiente pasamos a nuestro formulario 1 y agregamos el siguiente código para la conexión



En este caso ejecutamos la clase conexión que creamos antes, declaramos la variable conexión que almacenara un dato de nuestra clase creada, esta almacenara un dato que saquemos después de llamar a conexión() en nuestra clase por lo que se verificara el inicio de sesión, las credenciales y la conexión del servidor dándonos el siguiente mensaje



Con este mensaje verificamos que tenemos conexión con la base de datos por lo que agregar las líneas donde se verifican las credenciales de el servidor es indispensable al estar utilizando sql management studio con el inicio de sesión de Windows authentication ya que de esta forma no tenemos que agregar un usuario y contraseña al estar realizando las conexiones con el servidor de nuestras bases de datos, aunque podemos agregar un id y contraseña en la misma línea en caso de que no tengamos Windows authentication activo.

Conclusión.

En esta actividad repasamos los temas vistos en la materia de gestión de bases de datos ya que creamos una base de datos, distintas tablas y cada tabla con diferentes columnas, todas estas con sus respectivos datos, ya sean como enteros o cadenas de caracteres, además de agregar un disparador para la tabla de directivo donde al agregar a un empleado, este verificara si es directivo o no para agregarlo a la tabla de directivo así como el centro que supervisa, creamos el código de visual studio donde utilizaremos Windows forms y el lenguaje C# por lo que creamos una clase nueva donde podremos llamar en distintos formularios la conexión con la base de datos, visual studio agrega diferentes líneas de código analizando el código que escribimos para sugerir diferentes líneas de código aumentando así el rendimiento, además de analizar los errores que tengamos en el código y poder visualizar diferentes soluciones para este con distintas sugerencias, además de poder realizar diferentes aplicaciones para distintas plataformas.

Referencias.

Hackemi. (s. f.-b). *Hackemi/lenguajes_prog3*. GitHub.

https://github.com/Hackemi/lenguajes_prog3

Jocelyn Castellanos. (2019, 18 noviembre). *Conexion con SQL Server usando Windows Forms/Formularios de Windows* [Vídeo]. YouTube.

https://www.youtube.com/watch?v=tSTzkxyXR1o